

OE-EC604B — Operating System

Semester VI • 3L:0T:0P • 3 Credits

Module 1 Exam Notes: Introduction to Operating Systems

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

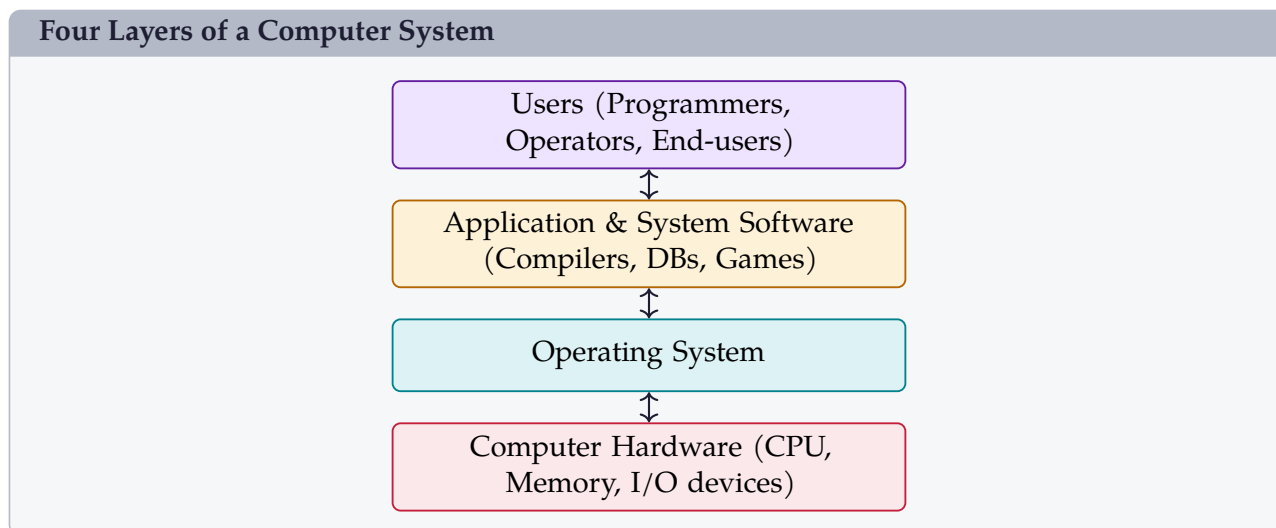
Contents

1	Operating System and Functions	2
1.1	Conceptual View of a Computer System	2
1.2	Core Functions of an Operating System	2
2	Evolution of Operating Systems	3
2.1	Batch Processing Systems	3
2.2	Interactive & Multi-Programmed Systems	3
2.3	Time-Sharing Systems	3
3	Specialized Operating Systems	3
3.1	Real-Time Systems (RTS)	3
3.2	Multi-Threading Systems	4

1. Operating System and Functions

An **Operating System (OS)** acts as an intermediary between the user of a computer and the computer hardware. Its primary goals are to make the computer system convenient to use and to execute user programs in an efficient manner.

1.1 Conceptual View of a Computer System



1.2 Core Functions of an Operating System

Definition: Operating System

An **Operating System** is a system software that manages computer hardware, software resources, and provides common services for computer programs.

From an engineering perspective, the functions of an OS can be classified into five major resource management categories:

Function	Description and Key Responsibilities
Process Management	Creation, execution, scheduling, synchronization, and termination of active processes.
Memory Management	Tracking free and allocated memory spaces, page mapping, swapping, and virtual memory allocation.
File System Management	Organising directories, files, mapping files to secondary storage, and access controls.
Device (I/O) Management	Buffering, caching, spooling, device drivers, and disk head scheduling.
Protection & Security	Controlling user access, authentication, preventing unauthorized access, and threat mitigation.

2. Evolution of Operating Systems

Operating systems have evolved from simple manual job setups on mainframes to complex distributed real-time systems.

2.1 Batch Processing Systems

Early mainframes used a batch system where the user had no direct access to the machine. Jobs were prepared offline on punched cards and grouped into batches by an operator.

- **Working Principle:** A program called the **resident monitor** stayed in memory, automatically loading and running subsequent jobs from the tape/card reader one after another.
- **Drawback:** No interactive user control, and extremely low CPU utilization due to the speed disparity between mechanical I/O devices (slow) and electronic CPU (fast).

2.2 Interactive & Multi-Programmed Systems

To overcome batch limitations, multi-programmed systems kept multiple jobs in memory simultaneously. When one job had to wait for I/O, the OS switched CPU execution to another job.

- **Interactive Systems:** Allowed direct communication between the user and system via a keyboard/terminal, giving immediate feedback.

2.3 Time-Sharing Systems

Time-sharing is a logical extension of multi-programming that provides fast interactive response times.

- **Working Principle:** The CPU executes multiple jobs by switching among them extremely fast using a fixed time quantum (slice). Each user gets a dedicated, albeit small, fraction of the CPU time, giving the illusion of a dedicated machine.
- **Key Goal:** Minimizing **response time** for multiple simultaneous users.

3. Specialized Operating Systems

3.1 Real-Time Systems (RTS)

A **Real-Time Operating System (RTOS)** is used when rigid time requirements are placed on the operation of a processor or the flow of data. The correctness of an RTOS depends not only on the logical result of computation but also on the **time** at which it is delivered.

Definition: Real-Time Systems

A **Real-Time System** guarantees that critical tasks are completed within strict, specified time constraints (deadlines).

Real-time systems are classified into two categories based on the severity of missing deadlines:

Feature	Hard Real-Time System	Soft Real-Time System
Deadline Violation	Catastrophic system failure.	Performance degradation only.
Tolerance	Zero tolerance. Deadlines must be met.	Occasional misses are tolerated.
Memory Support	Virtual memory is usually disabled.	Virtual memory is allowed.
Examples	pacemakers, missile guidance, airbags.	Video streaming, online gaming, audio.

3.2 Multi-Threading Systems

A **thread** (or lightweight process) is a basic unit of CPU utilization. In a multi-threading operating system, a single process can spawn multiple concurrent threads of execution, all sharing the same code segment, data segment, and system resources, but having their own thread ID, program counter, register set, and stack.

- **Benefits:** Responsiveness (non-blocking UI), resource sharing (threads share process memory), economy (context switching is faster than process creation), and multicore utilization.

Quick Revision — Module 1

Introduction to Operating Systems | OE-EC604B

Key Evolution Properties at a Glance

Batch OS	Low CPU utilization, high turnaround time, offline execution.
Multiprogramming	High CPU utilization, keeps multiple jobs in memory, switches on I/O.
Time-Sharing	Fixed time-slice (quantum), interactive, minimizes response time.
Hard Real-Time	Strict deadlines, safety-critical, no virtual memory, predictable delay.
Soft Real-Time	Best-effort deadlines, priority scheduling, virtual memory allowed.

One-Line Definitions

- **Operating System:** System software managing hardware resources and acting as a user-hardware interface.
- **Spooling:** Simultaneous Peripheral Operations On-Line; placing I/O data in disk buffer to speed up batch execution.
- **Multiprogramming:** OS technique running multiple jobs concurrently to maximize CPU utilization.
- **Time-Sharing:** Preemptive multiprogramming where CPU switches quickly between users using a time slice.
- **Real-Time OS:** Time-bound operating system designed for deterministic response times and deadlines.
- **Hard Real-Time:** OS where missing a single task deadline results in catastrophic system failure.
- **Soft Real-Time:** OS where deadlines are preferred but occasional misses do not cause system failure.
- **Thread:** A lightweight process representing a single sequential flow of control within a process.
- **Resident Monitor:** Early batch supervisor program residing in memory to load jobs automatically.

Frequently Asked / Viva Questions

Q1. What are the primary objectives of an operating system?

To make the computer system convenient to use for the user, and to allocate and

Frequently Asked / Viva Questions (contd.)

manage the hardware resources (CPU, memory, devices) in an efficient and fair manner.

Q2. Explain the difference between Multiprogramming and Multiprocessing.

Multiprogramming keeps multiple processes in main memory simultaneously on a single CPU, switching between them to keep the CPU busy. Multiprocessing involves a computer system with two or more physical CPUs executing processes in parallel.

Q3. What is spooling? How does it differ from buffering?

Spooling uses the disk as a large buffer to store incoming/outgoing jobs and data for slow peripheral devices (like printers), enabling concurrent CPU processing. Buffering is a temporary storage area in main memory used during I/O transfers to smooth out speed differences.

Q4. Differentiate between Hard Real-Time and Soft Real-Time systems.

Hard Real-Time systems have strict, non-negotiable deadlines where any failure causes catastrophic system collapse (e.g., flight controller). Soft Real-Time systems treat deadlines as highly desirable but tolerate occasional misses without failure (e.g., video player).

Q5. What is the time-sharing concept? Which scheduling is typically used?

Time-sharing allows multiple users to share a single CPU interactively. The OS allocates a small time slice (quantum) to each user process and switches preemptively using Round Robin scheduling to maintain low response times.

Q6. List the primary resource management functions of an OS.

Process management, Memory management, File system management, I/O device management, and Protection/Security management.

Q7. Why is virtual memory usually not supported in hard real-time systems?

Virtual memory introduces unpredictable time delays due to page faults and disk access latency, which violates the strict deterministic timing requirements of hard real-time systems.

Q8. Explain how multithreading improves application responsiveness.

By splitting execution into multiple threads, an application can execute background tasks (like disk writing or computations) in one thread while the main thread keeps responding to user actions (like button clicks) without freezing the UI.

Q9. What is the role of a resident monitor in early batch systems?

It was the first operating program. It resided in main memory and automatically handled job transitions by reading the next job card/tape as soon as the current one finished, reducing setup time.

Q10. Why is CPU scheduling required in multiprogrammed systems?

Frequently Asked / Viva Questions (contd.)

When the active process performs I/O, it must wait. To prevent the CPU from sitting idle, CPU scheduling selects another ready process from memory to execute, maximizing CPU utilization.

Comparative Analysis of Operating Systems

System Type	Primary Goal	CPU Utilization	Interaction Level
Batch	Maximize Throughput	Low to Moderate	None (offline jobs)
Multiprogramming	Maximize CPU usage	High (context switches on I/O)	Low (background jobs)
Time-Sharing	Minimize Response Time	Moderate to High	Very High (interactive)
Real-Time	Guarantee Deadline	Secondary concern	Low (automated control)